

DOCKER_TASK_2

Tasks:

- ❖ Create a tomcat container on 8080 and deploy sample application in tomcat.
- ❖ Create volume and deploy tomcat container on port 8081.
- ❖ Limit the nginx container to 500 MB.
- ❖ Create a sample docker file using below instructions.
- ❖ Base module as amazonlinux:latest
- ❖ Maintainer you name
- ❖ Install nginx
- ❖ COPY one index.html file to image
- ❖ Expose on port 80
- ❖ Command to start the nginx container
- ❖ Push image to Docker hub
- ❖ push image to AWS ECR

1. Create a tomcat container on 8080 and deploy sample application in tomcat.

→ docker pull tomcat:latest

```
aws Search [Alt+S] United States (N. Virginia) Ec2 (0607-9593-281)

root@ip-172-31-27-214:~# docker pull tomcat:latest
latest: Pulling from library/tomcat
3629ac5b9f4: Already exists
d41d65c3828: Pull complete
be760c2a9a8: Extracting [=====>] 41.78MB/92.17MB
f4fb700ef54: Download complete
0702afe109d: Download complete
99c7608407b: Download complete
ca1458a9031: Download complete
```

→ docker container run -itd --name tomcat_test -p 8080:8080 tomcat:latest

```
root@ip-172-31-27-214:~# docker container run -itd --name tomcat_test -p 8080:8080 tomcat:latest
21f0f61143595fa818308134b5f5cead9de77e1f54b73098db98d35e2alc8cfd
root@ip-172-31-27-214:~# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS
21f0f6114359   tomcat:latest "catalina.sh run"       About a minute ago Up About a minute 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
root@ip-172-31-27-214:~#
```

→ go inside the container “docker exec -it container_id /bin/sh

→ Then go webapps and check for the file

```
root@ip-172-31-27-214:~# docker exec -it 21f0f6114359 /bin/sh
# pwd
/usr/local/tomcat
# ls
bin          conf          filtered-KEYS  LICENSE  native-jni-lib  README.md  RUNNING.txt  upstream-KEYS  webapps.dist
BUILDING.txt  CONTRIBUTING.md  lib          logs     NOTICE         RELEASE-NOTES  temp        webapps       work
# cd webapps
# ls
# pwd
/usr/local/tomcat/webapps
#
```

→ wget sample.html from webaspps

→update using apt update -y

→apt install wget -y

```
/bin/sh: 8: wget: not found
# apt update
apt install wget -y
Get:1 http://archive.ubuntu.com/ubuntu noble InRelease [256 kB]
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:5 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Packages [34.8 kB]
Get:6 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [117 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [331 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [3,061 kB]
Get:9 http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages [19.3 MB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1,199 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1,802 kB]
Get:12 http://archive.ubuntu.com/ubuntu noble/main amd64 Packages [1,808 kB]
Get:13 http://archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Packages [38.1 kB]
Get:14 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1,978 kB]
```

→now run wget and tomcat sample_link

```
Selecting previously unselected package libpsl5t64:amd64.
(Reading database ... 6571 files and directories currently installed.)
Preparing to unpack .../libpsl5t64_0.21.2-1.1build1_amd64.deb ...
Unpacking libpsl5t64:amd64 (0.21.2-1.1build1) ...
Selecting previously unselected package publicsuffix.
Preparing to unpack .../publicsuffix_20231001.0357-0.1_all.deb ...
Unpacking publicsuffix (20231001.0357-0.1) ...
Selecting previously unselected package wget.
Preparing to unpack .../wget_1.21.4-1ubuntu4.1_amd64.deb ...
Unpacking wget (1.21.4-1ubuntu4.1) ...
Setting up libpsl5t64:amd64 (0.21.2-1.1build1) ...
Setting up publicsuffix (20231001.0357-0.1) ...
Setting up wget (1.21.4-1ubuntu4.1) ...
Processing triggers for libc-bin (2.39-0ubuntu8.6) ...
# wget https://tomcat.apache.org/tomcat-9.0-doc/appdev/sample/sample.war
--2026-02-05 10:55:11-- https://tomcat.apache.org/tomcat-9.0-doc/appdev/sample/sample.war
Resolving tomcat.apache.org (tomcat.apache.org)... 151.101.2.132, 2a04:4e42::644
Connecting to tomcat.apache.org (tomcat.apache.org)|151.101.2.132|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4606 (4.5K)
Saving to: 'sample.war'

sample.war      100%[=====>]  4.50K  --.-KB/s  in 0s

2026-02-05 10:55:11 (56.5 MB/s) - 'sample.war' saved [4606/4606]
```

→ls

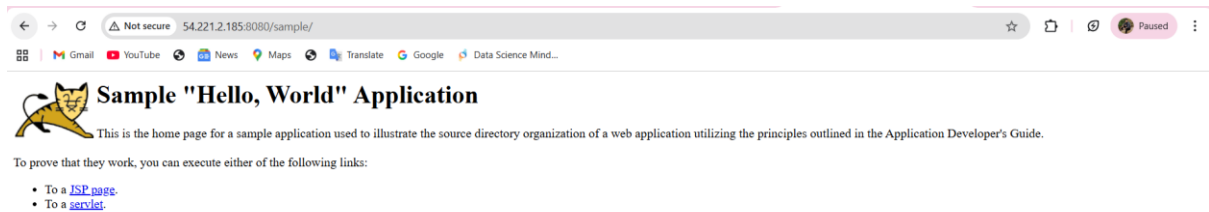
```
2026-02-05 10:55:11 (56.5 MB/s) - 'sample.war' saved [4606/4606]

# ls
sample sample.war
#
```

i-028f1dc9ea91e8b89 (Docker_controller)

PublicIPs: 54.221.2.185 PrivateIPs: 172.31.27.214

→Now check the status



← → ↻ 🔒 Not secure 54.221.2.185:8080/sample/ ☆ 📄 🔄 ⏸ Paused ⋮

📧 Gmail 📺 YouTube 🗺 Maps 🗨 Translate 🌐 Google 📊 Data Science Mind...

Sample "Hello, World" Application

This is the home page for a sample application used to illustrate the source directory organization of a web application utilizing the principles outlined in the Application Developer's Guide.

To prove that they work, you can execute either of the following links:

- To a [JSP page](#).
- To a [servlet](#).

2. Create volume and deploy tomcat container on port 8081.

→docker volume create deployment

→docker volume ls

→cd /var/lib/docker/volumes/deployment/_data#

→ll

```

drwx-----x 3 root root 4096 Feb  5 11:07 ../
root@ip-172-31-27-214:/var/lib/docker/volumes/deployment/_data# cd ../
root@ip-172-31-27-214:/var/lib/docker/volumes/deployment/_data# ll
total 8
drwxr-xr-x 2 root root 4096 Feb  5 11:07 ./
drwx-----x 3 root root 4096 Feb  5 11:07 ../
root@ip-172-31-27-214:/var/lib/docker/volumes/deployment/_data# cd ..
root@ip-172-31-27-214:/var/lib/docker/volumes/deployment# cd ../
root@ip-172-31-27-214:/var/lib/docker/volumes# ll
total 48
drwx-----x 6 root root 4096 Feb  5 11:07 ./
drwx--x--x 12 root root 4096 Feb  5 09:25 ../
drwx-----x 3 root root 4096 Feb  4 12:56 858620b7e918d82aa7a3a3a685cc29240b22b7c1adffe812e5d4c2d80ad8e7aa/
drwx-----x 3 root root 4096 Feb  4 13:21 b682d9e6000f4a981e4a446186396f5414443fbd7b14b2891a8208124bea931/
drw----- 1 root root 202, 1 Feb  5 09:25 backingFsBlockDev
drwx-----x 3 root root 4096 Feb  5 11:07 deployment/
drwx-----x 3 root root 4096 Feb  4 12:59 f220ab6ceb9409d6e5628f73dfbffa22154447448d8a6459cd6cal74c8272be/
-rw----- 1 root root 32768 Feb  5 11:07 metadata.db
root@ip-172-31-27-214:/var/lib/docker/volumes/deployment# ls
_data
root@ip-172-31-27-214:/var/lib/docker/volumes/deployment# cd _data/
root@ip-172-31-27-214:/var/lib/docker/volumes/deployment/_data# ll
total 8
drwxr-xr-x 2 root root 4096 Feb  5 11:07 ./
drwx-----x 3 root root 4096 Feb  5 11:07 ../
root@ip-172-31-27-214:/var/lib/docker/volumes/deployment/_data#

```

i-028f1dc9ea91e8b89 (Docker_controller)

PublicIPs: 54.221.2.185 PrivateIPs: 172.31.27.214

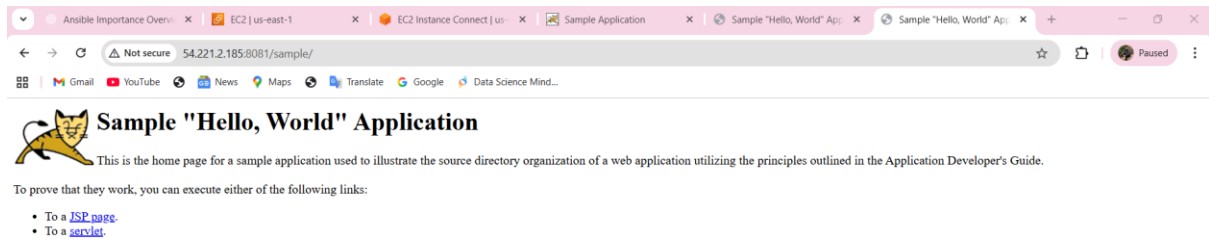
→ docker container run -itd -p 8081:8080 -v /var/lib/docker/volumes/deployment/_data:/usr/local/tomcat/webapps tomcat:latest

```

root@ip-172-31-27-214:~# docker container run -itd -p 8081:8080 -v /var/lib/docker/volumes/deployment/_data:/usr/local/tomcat/webapps tomcat:latest
0d834451a507d3950daaafdefc4a624b6d0f21852125002d664f5256cdf905e8
root@ip-172-31-27-214:~# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
0d834451a507   tomcat:latest  "catalina.sh run"        About a minute ago    Up About a minute    0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp    youthful_lehmann
21f0f6114359   tomcat:latest  "catalina.sh run"        About an hour ago     Up About an hour     0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp    tomcat_test
root@ip-172-31-27-214:~#

```

→ check for the deployment



3. Limit the nginx container to 500 MB.

→ docker run -d --name nginx500 -p 80:80 --memory="500m" nginx

```
root@ip-172-31-27-214:~# docker run -d --name nginx500 -p 80:80 --memory="500m" nginx
```

→ docker stats

```

CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O      PIDS
05d7f0d4a31a   nginx500  0.00%    3.383MiB / 500MiB   0.68%    656B / 126B    0B / 8.19kB     3
0d834451a507   youthful_lehmann  0.05%    94.49MiB / 7.752GiB  1.19%    5.04kB / 6.37kB  0B / 319kB     35
21f0f6114359   tomcat_test  0.06%    139.2MiB / 7.752GiB  1.75%    36.6MB / 194kB   36MB / 98.2MB   35

```

4. Create a sample docker file using below instructions.

5. Base module as amazonlinux:latest

6. Maintainer you name

7. Install nginx
8. COPY one index.html file to image
9. Expose on port 80
10. Command to start the nginx container

→ Create a dockerfile and do below tasks

1. Base module as amazonlinux:latest. Task 5
2. Maintainer you name task 6
3. Install nginx task 7
4. COPY one index.html file to image. Task 8
5. Expose on port 80 task 9
6. Command to start the nginx container task 10

→ Step1: mkdir nginx-task

→ cd nginx-task

→ vi index.html

```
root@ip-172-31-27-214:~# mkdir nginx-task
cd nginx-task
root@ip-172-31-27-214:~/nginx-task# pwd
/root/nginx-task
root@ip-172-31-27-214:~/nginx-task# ls
root@ip-172-31-27-214:~/nginx-task# vi index.html
```

```
root@ip-172-31-27-214:~# mkdir nginx-task
cd nginx-task
root@ip-172-31-27-214:~/nginx-task# pwd
/root/nginx-task
root@ip-172-31-27-214:~/nginx-task# ls
root@ip-172-31-27-214:~/nginx-task# vi index.html
root@ip-172-31-27-214:~/nginx-task# cat index.html
<h1>Hello from Arjumand</h1>
root@ip-172-31-27-214:~/nginx-task#
```

→ vi Dockerfile

```
aws [Alt+S] Ask Amazon Q United States (N. Virginia) Ec2

#5.Base module as amazonlinux:latest
FROM amazonlinux:latest
#6.Maintainer you name
MAINTAINER ArjumandMohiuddin
#7.Install nginx
RUN yum update -y && yum install -y nginx
#8.COPY one index.html file to image
COPY index.html /usr/share/nginx/html/index.html
#9.Expose on port 80
EXPOSE 80
#10.Command to start the nginx container
CMD ["nginx", "-g", "daemon off;"]
```

→ Step 4: Build Docker Image

→ docker run -itd --name arj-nginx -p 9090:80 arj-nginx:1.0

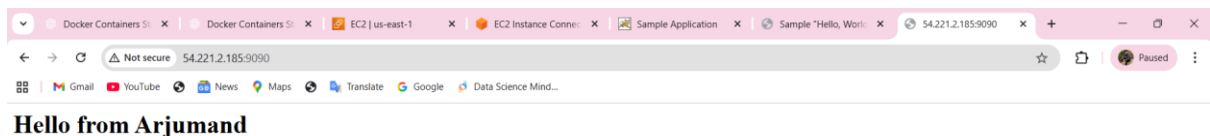
```
---> Using cache
---> cffb964f9c1
Step 4/6 : COPY index.html /usr/share/nginx/html/index.html
---> Using cache
---> 041549f77d64
Step 5/6 : EXPOSE 80
---> Using cache
---> af5197b6be1e
Step 6/6 : CMD ["nginx", "-g", "daemon off;"]
---> Using cache
---> 5a325d81cc87
Successfully built 5a325d81cc87
Successfully tagged arj-nginx:1.0
root@ip-172-31-27-214:~/nginx-task# docker run -d -p 80:80 --name nginx-container amazon-nginx
Unable to find image 'amazon-nginx:latest' locally
docker: Error response from daemon: pull access denied for amazon-nginx, repository does not exist or may require 'docker login': denied: requested access to the resource is denied

Run 'docker run --help' for more information
root@ip-172-31-27-214:~/nginx-task# docker run -itd --name arj-nginx -p 9090:80 arj-nginx:1.0
8648f9f93b75449ca505edc7efcb9a43f0d02b36e5e9dela174a0d1223cb6b1b
root@ip-172-31-27-214:~/nginx-task# docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS                               NAMES
8648f9f93b75        arj-nginx:1.0      "nginx -g 'daemon of..."   18 seconds ago      Up 18 seconds      0.0.0.0:9090->80/tcp, [::]:9090->80/tcp   arj-nginx
0d834451a507        tomcat:latest      "catalina.sh run"      2 hours ago         Up 2 hours         0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp   youthful_lehmann
21f0f6114359        tomcat:latest      "catalina.sh run"      3 hours ago         Up 3 hours         0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   tomcat_test

root@ip-172-31-27-214:~/nginx-task#

i-028f1dc9ea91e8b89 (Docker_controller)
PublicIPs: 54.221.2.185  PrivateIPs: 172.31.27.214
```

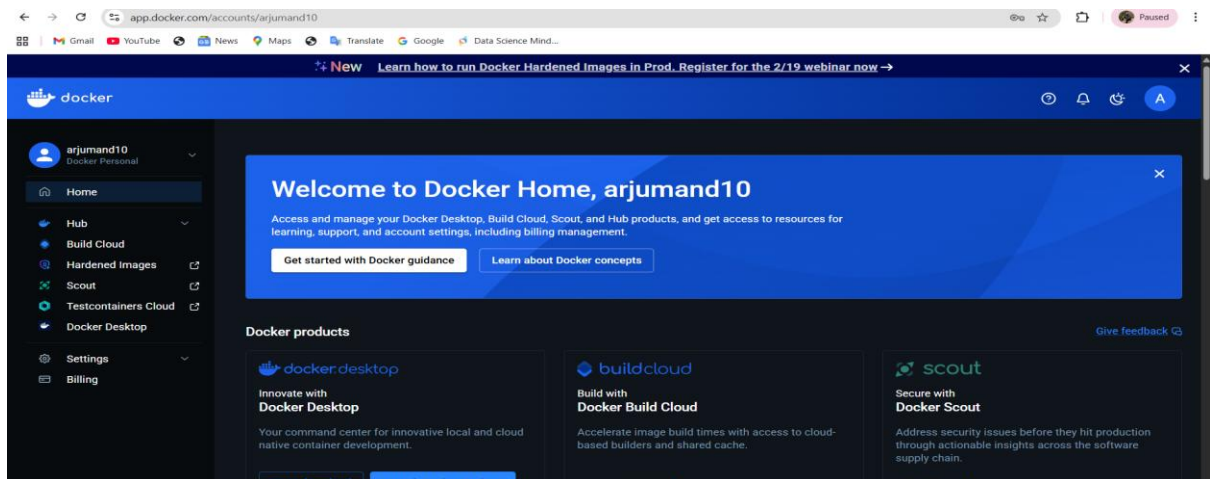
→ verify it by entering ip with the port number which we allocated



11. Push image to Docker hub

→ Create a docker account sign up and login

→ docker tag arj-nginx:1.0 arjumand10/arj-nginx:1.0



→ Login to docker then one time pass word will be provided add that in path which is given

```
Your one-time device confirmation code is: BSLL-NNNN
Press ENTER to open your browser or submit your device code here: https://login.docker.com/activate

Waiting for authentication in the browser...

WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
root@ip-172-31-27-214:~/nginx-task#

i-028f1dc9ea91e8b89 (Docker_controller)
PublicIPs: 54.221.2.185  PrivateIPs: 172.31.27.214
```

→ docker push arjumand10/arj-nginx:1.0

```
root@ip-172-31-27-214:~/nginx-task# docker push arjumand10/arj-nginx:1.0
The push refers to repository [docker.io/arjumand10/arj-nginx]
81a1e3ee91f5: Pushed
7bb3c0887309: Pushed
b535d73979ee: Mounted from library/amazonlinux
1.0: digest: sha256:ec12d9c0f088167216a3a3cb315dcd4fd75c1b06b2526328d1f7aa0cad60c737 size: 949
root@ip-172-31-27-214:~/nginx-task#
```

i-028f1dc9ea91e8b89 (Docker_controller)

PublicIPs: 54.221.2.185 PrivateIPs: 172.31.27.214

→ Image is successfully pushed



12. push image to AWS ECR

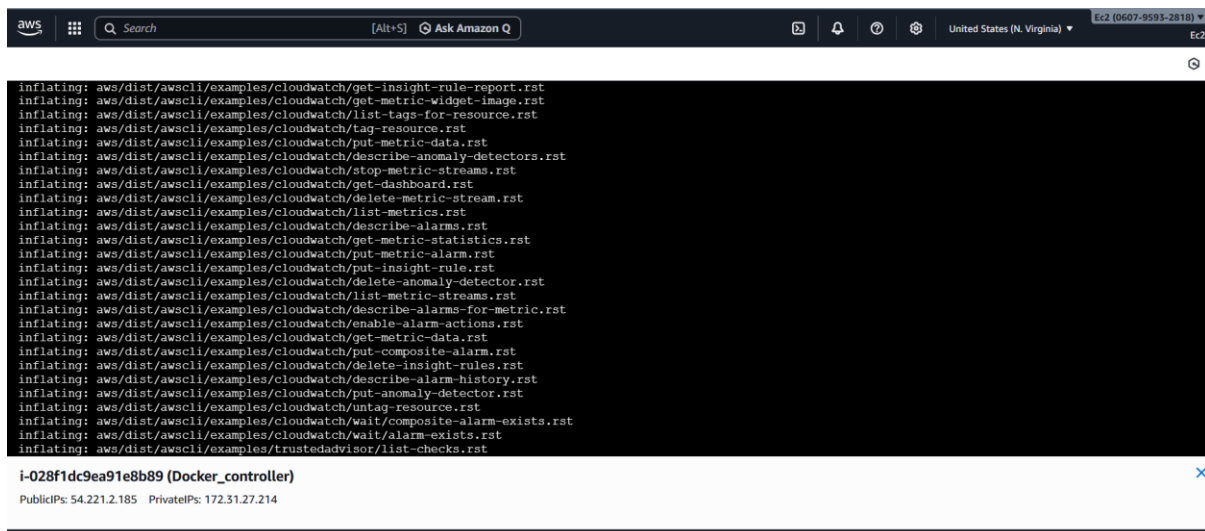
→ sudo apt update

→ sudo apt install -y unzip curl

→ curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"

→ unzip awscliv2.zip

→ sudo ./aws/install



```
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/licenses/AUTHORS.rst
You can now run: /usr/local/bin/aws --version
root@ip-172-31-27-214:~# aws --version
aws-cli/2.33.15 Python/3.12.11 Linux/6.14.0-1018-aws exe/x86_64.ubuntu.24
root@ip-172-31-27-214:~#
```

i-028f1dc9ea91e8b89 (Docker_controller)

PublicIPs: 54.221.2.185 PrivateIPs: 172.31.27.214

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

→ AWS configure

```
aws
```

Search [Alt+S] Ask Amazon Q

United States (N. Virginia) EC2 (0607-9593-2818)

```
root@ip-172-31-27-214:~# aws configure
AWS Access Key ID [None]: AKIAQ4J5YECJMH7UELPN
AWS Secret Access Key [None]: PNg6N97F0UjH1+FVSaOmHdH0fq+C3S8emQR5e0g
Default region name [None]: us-east-1
Default output format [None]: json
root@ip-172-31-27-214:~# aws s3 ls
```

→ aws ecr create-repository --repository-name arj-nginx --region us-east-1

```
root@ip-172-31-27-214:~# aws ecr create-repository --repository-name arj-nginx --region us-east-1
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:us-east-1:060795932818:repository/arj-nginx",
    "registryId": "060795932818",
    "repositoryName": "arj-nginx",
    "repositoryUri": "060795932818.dkr.ecr.us-east-1.amazonaws.com/arj-nginx",
    "createdAt": "2026-02-05T14:20:05.972000+00:00",
    "imageTagMutability": "MUTABLE",
    "imageScanningConfiguration": {
      "scanOnPush": false
    },
    "encryptionConfiguration": {
      "encryptionType": "AES256"
    }
  }
}
root@ip-172-31-27-214:~#
```

i-028f1dc9ea91e8b89 (Docker_controller)

PublicIPs: 54.221.2.185 PrivateIPs: 172.31.27.214

→ Verify, go to aws ecr

Amazon ECR > Private registry > Repositories

Amazon Elastic Container Service

Private registries

Repositories

Features & Settings

Public registry

Repositories

Settings

Private repositories (1)

Search by repository substring

	Repository name	URI	Created at	Tag immutability	Encryption type
☐	arj-nginx	060795932818.dkr.ecr.us-east-1.amazonaws.com/arj-nginx	February 05, 2026, 19:50:05 (UTC+05.5)	Mutable	AES-256

→ aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 060795932818.dkr.ecr.us-east-1.amazonaws.com

```
root@ip-172-31-27-214:~# aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 060795932818.dkr.ecr.us-east-1.amazonaws.com
WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/
Login Succeeded
root@ip-172-31-27-214:~#
```

i-028f1dc9ea91e8b89 (Docker_controller)

PublicIPs: 54.221.2.185 PrivateIPs: 172.31.27.214

→ docker tag arj-nginx:1.0 060795932818.dkr.ecr.us-east-1.amazonaws.com/arj-nginx:1.0

```
root@ip-172-31-27-214:~# docker tag arj-nginx:1.0 060795932818.dkr.ecr.us-east-1.amazonaws.com/arj-nginx:1.0
root@ip-172-31-27-214:~#
```

i-028f1dc9ea91e8b89 (Docker_controller)

PublicIPs: 54.221.2.185 PrivateIPs: 172.31.27.214

→ docker push 060795932818.dkr.ecr.us-east-1.amazonaws.com/arj-nginx:1.0

```
https://docs.docker.com/go/credential-store/

Login Succeeded
root@ip-172-31-27-214:~# docker tag arj-nginx:1.0 060795932818.dkr.ecr.us-east-1.amazonaws.com/arj-nginx:1.0
root@ip-172-31-27-214:~# docker push 060795932818.dkr.ecr.us-east-1.amazonaws.com/arj-nginx:1.0
The push refers to repository [060795932818.dkr.ecr.us-east-1.amazonaws.com/arj-nginx]
31ale3ce51f3: Pushed
7bb3c0887309: Pushing [----->] 110.8MB/185.6MB
b535d73979ee: Pushing [----->] 91.75MB/148.6MB

i-028f1dc9ea91e8b89 (Docker_controller)
PublicIPs: 54.221.2.185 PrivateIPs: 172.31.27.214
```

→Verify, go to ecr in aws and select the repository then we will see.

